

Setup and imports

```
In [76]: import os
import random
import torch
import torch.nn as nn
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from torch.utils.data import DataLoader, TensorDataset
from torchvision import datasets, transforms, models
from sklearn.metrics import (accuracy_score, precision_score, recall_score,
                             confusion_matrix)

SEED = 17

random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)
torch.cuda.manual_seed(SEED)

dv = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Using device: {dv}")

if torch.cuda.is_available():
    print(f"GPU: {torch.cuda.get_device_name(0)}")

DATASET_DIR = "./pets-facial-expression-dataset"

assert os.path.exists(DATASET_DIR), f"Dataset directory '{DATASET_DIR}' does not exist"

# Split ratios
TRAIN_RATIO = 0.70
VALID_RATIO = 0.15
TEST_RATIO = 0.15

# hyperparameters
BATCH_SIZE = 32
IMAGE_SIZE = 224
NUM_WORKERS = 0 # Adjust based on your system's capabilities
```

Using device: cuda

GPU: NVIDIA GeForce RTX 3060 Ti

Prepare Datasets

```
In [77]: train_transform = transforms.Compose([
    transforms.Resize((IMAGE_SIZE, IMAGE_SIZE)),
    transforms.RandAugment(num_ops=10, magnitude=3), # Apply BEFORE ToTensor
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
])

val_test_transform = transforms.Compose([
    transforms.Resize((IMAGE_SIZE, IMAGE_SIZE)),
```

```

transforms.ToTensor(),
transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0
])

```

Load Datasets

```

In [78]: # Custom Subset class to avoid transform leakage
class CustomSubset(torch.utils.data.Dataset):
    def __init__(self, dataset, indices, transform=None):
        self.dataset = dataset
        self.indices = indices
        self.transform = transform if transform is not None else transform

    def __getitem__(self, idx):
        image, label = self.dataset[self.indices[idx]]
        # Always apply transform - guaranteed to return tensor
        image = self.transform(image)
        return image, label

    def __len__(self):
        return len(self.indices)

# Load all data WITHOUT transforms (need PIL images for RandAugment)
full_dataset = datasets.ImageFolder(root=DATASET_DIR, transform=None)
class_names = full_dataset.classes
num_classes = len(class_names)
print(f"Classes: {class_names}")
print(f"Number of classes: {num_classes}")
print(f"Total samples: {len(full_dataset)}")

# Create 70/15/15 split
total_size = len(full_dataset)
train_size = int(total_size * TRAIN_RATIO)
valid_size = int(total_size * VALID_RATIO)
test_size = total_size - train_size - valid_size

# Create indices for random split
generator = torch.Generator().manual_seed(SEED)
indices = torch.randperm(total_size, generator=generator).tolist()
train_indices = indices[:train_size]
val_indices = indices[train_size:train_size + valid_size]
test_indices = indices[train_size + valid_size:]

# Use CustomSubset to apply transforms independently (NO LEAKAGE)
train_dataset = CustomSubset(full_dataset, train_indices, transform=train_transform)
val_dataset = CustomSubset(full_dataset, val_indices, transform=val_transform)
test_dataset = CustomSubset(full_dataset, test_indices, transform=test_transform)

print(f"\nDataset split:")
print(f"  Training: {len(train_dataset)} samples ({TRAIN_RATIO*100:.0f}%)")
print(f"  Validation: {len(val_dataset)} samples ({VALID_RATIO*100:.0f}%)")
print(f"  Test: {len(test_dataset)} samples ({TEST_RATIO*100:.0f}%)")

```

```
Classes: ['Angry', 'Other', 'Sad', 'happy']  
Number of classes: 4  
Total samples: 1000
```

```
Dataset split:  
  Training: 700 samples (70%)  
  Validation: 150 samples (15%)  
  Test: 150 samples (15%)
```

Create DataLoaders

```
In [79]: train_dataloader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True)  
val_dataloader = DataLoader(val_dataset, batch_size=BATCH_SIZE, shuffle=False)  
test_dataloader = DataLoader(test_dataset, batch_size=BATCH_SIZE, shuffle=False)  
  
print("\n DataLoaders created successfully!")  
  
# count samples per class in training dataset  
def count_samples_per_class(dataset, class_names):  
    class_counts = {class_name: 0 for class_name in class_names}  
    for _, label in dataset:  
        class_counts[class_names[label]] += 1  
    return class_counts  
  
train_class_counts = count_samples_per_class(train_dataset, class_names)  
print("\nTraining dataset class distribution:")  
for class_name, count in train_class_counts.items():  
    print(f"{class_name}: {count} samples")  
  
val_class_counts = count_samples_per_class(val_dataset, class_names)  
print("\nValidation dataset class distribution:")  
for class_name, count in val_class_counts.items():  
    print(f"{class_name}: {count} samples")  
  
test_class_counts = count_samples_per_class(test_dataset, class_names)  
print("\nTest dataset class distribution:")  
for class_name, count in test_class_counts.items():  
    print(f"{class_name}: {count} samples")
```

DataLoaders created successfully!

Training dataset class distribution:
Angry: 177 samples
Other: 174 samples
Sad: 179 samples
happy: 170 samples

Validation dataset class distribution:
Angry: 34 samples
Other: 40 samples
Sad: 33 samples
happy: 43 samples

Test dataset class distribution:
Angry: 39 samples
Other: 36 samples
Sad: 38 samples
happy: 37 samples

View Samples

```
In [80]: def denormalise(image_tensor):
    mean = torch.tensor([0.485, 0.456, 0.406]).view(3, 1, 1)
    std = torch.tensor([0.229, 0.224, 0.225]).view(3, 1, 1)
    image_tensor = image_tensor.cpu() # Ensure the tensor is on CPU for
    image_tensor = image_tensor * std + mean # Denormalize
    image_tensor = torch.clamp(image_tensor, 0, 1) # Clamp values to [0,
    return image_tensor

def show_batch(dataloader, class_names, num_images=8):
    images, labels = next(iter(dataloader))
    images = images[:num_images]
    labels = labels[:num_images]

    images = [denormalise(img) for img in images]

    plt.figure(figsize=(12, 8))
    for i in range(num_images):
        plt.subplot(4, 4, i + 1)
        plt.imshow(images[i].permute(1, 2, 0)) # Convert from (C, H, W)
        plt.title(class_names[labels[i]])
        plt.axis('off')
    plt.tight_layout()
    plt.show()

print("\nShowing a batch of training images:")
show_batch(train_dataloader, class_names)
```

Showing a batch of training images:



Dataset Analysis & Diagnostics

```
In [81]: # Verify data loading and class distribution
print("="*60)
print("DATASET DIAGNOSTICS")
print("="*60)

print("\n1. Dataset Sizes:")
print(f"    Total images: {total_size}")
print(f"    Train: {len(train_dataset)} | Val: {len(val_dataset)} | Test: ")

print("\n2. Training Set Statistics:")
```

```
print(f"    Total training samples: {len(train_dataset)}")
for cls, count in train_class_counts.items():
    pct = (count / len(train_dataset)) * 100
    print(f"        {cls}: {count:3d} samples ({pct:5.1f}%)")

print("\n3. Sample shapes and value ranges:")
sample_batch = next(iter(train_dataloader))
images, labels = sample_batch
print(f"    Batch shape: {images.shape}")
print(f"    Min pixel value: {images.min():.4f}")
print(f"    Max pixel value: {images.max():.4f}")
print(f"    Mean pixel value: {images.mean():.4f}")
print(f"    Std pixel value: {images.std():.4f}")

print("\n4. Label distribution in batch:")
for i, label in enumerate(labels[:8]):
    print(f"    Image {i}: {class_names[label]}")

print("\n5. Training iterations per epoch:")
print(f"    Train batches: {len(train_dataloader)}")
print(f"    Val batches: {len(val_dataloader)}")
print(f"    Test batches: {len(test_dataloader)}")
print("\n" + "="*60)
```

```
=====
DATASET DIAGNOSTICS
=====
```

1. Dataset Sizes:
Total images: 1000
Train: 700 | Val: 150 | Test: 150
2. Training Set Statistics:
Total training samples: 700
Angry: 177 samples (25.3%)
Other: 174 samples (24.9%)
Sad: 179 samples (25.6%)
happy: 170 samples (24.3%)
3. Sample shapes and value ranges:
Batch shape: torch.Size([32, 3, 224, 224])
Min pixel value: -2.1179
Max pixel value: 2.6400
Mean pixel value: 0.0247
Std pixel value: 1.3510
4. Label distribution in batch:
Image 0: Angry
Image 1: Sad
Image 2: happy
Image 3: Sad
Image 4: Angry
Image 5: Other
Image 6: happy
Image 7: Angry
5. Training iterations per epoch:
Train batches: 22
Val batches: 5
Test batches: 5

```
=====
```

VGG16

```
In [82]: class VGG16(nn.Module): # vgg16 configuration C
    def __init__(self, num_classes=4):
        super(VGG16, self).__init__()
        self.block1 = nn.Sequential(
            nn.Conv2d(3, 64, kernel_size=3, stride=1, padding=1),
            nn.BatchNorm2d(64),
            nn.ReLU(inplace=True),
            nn.Conv2d(64, 64, kernel_size=3, stride=1, padding=1),
            nn.BatchNorm2d(64),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=2, stride=2)
        )
        self.block2 = nn.Sequential(
            nn.Conv2d(64, 128, kernel_size=3, stride=1, padding=1),
            nn.BatchNorm2d(128),
            nn.ReLU(inplace=True),
            nn.Conv2d(128, 128, kernel_size=3, stride=1, padding=1),
```

```
        nn.BatchNorm2d(128),
        nn.ReLU(inplace=True),
        nn.MaxPool2d(kernel_size=2, stride=2)
    )
    self.block3 = nn.Sequential(
        nn.Conv2d(128,256, kernel_size=3, stride=1, padding=1),
        nn.BatchNorm2d(256),
        nn.ReLU(inplace=True),
        nn.Conv2d(256,256, kernel_size=3, stride=1, padding=1),
        nn.BatchNorm2d(256),
        nn.ReLU(inplace=True),
        nn.Conv2d(256,256, kernel_size=3, stride=1, padding=1),
        nn.BatchNorm2d(256),
        nn.ReLU(inplace=True),
        nn.MaxPool2d(kernel_size=2, stride=2)
    )
    self.block4 = nn.Sequential(
        nn.Conv2d(256,512, kernel_size=3, stride=1, padding=1),
        nn.BatchNorm2d(512),
        nn.ReLU(inplace=True),
        nn.Conv2d(512,512, kernel_size=3, stride=1, padding=1),
        nn.BatchNorm2d(512),
        nn.ReLU(inplace=True),
        nn.Conv2d(512,512, kernel_size=3, stride=1, padding=1),
        nn.BatchNorm2d(512),
        nn.ReLU(inplace=True),
        nn.MaxPool2d(kernel_size=2, stride=2)
    )
    self.block5 = nn.Sequential(
        nn.Conv2d(512,512, kernel_size=3, stride=1, padding=1),
        nn.BatchNorm2d(512),
        nn.ReLU(inplace=True),
        nn.Conv2d(512,512, kernel_size=3, stride=1, padding=1),
        nn.BatchNorm2d(512),
        nn.ReLU(inplace=True),
        nn.Conv2d(512,512, kernel_size=3, stride=1, padding=1),
        nn.BatchNorm2d(512),
        nn.ReLU(inplace=True),
        nn.MaxPool2d(kernel_size=2, stride=2)
    )

    self.features = nn.Sequential(
        self.block1,
        self.block2,
        self.block3,
        self.block4,
        self.block5
    )

    self.classifier = nn.Sequential(
        nn.Flatten(),
        nn.Linear(512*7*7, 4096),
        nn.BatchNorm1d(4096),
        nn.ReLU(inplace=True),
        nn.Dropout(p=0.5),
        nn.Linear(4096, 4096),
        nn.BatchNorm1d(4096),
        nn.ReLU(inplace=True),
        nn.Dropout(p=0.5),
        nn.Linear(4096, num_classes)
```

```

    )
    def forward(self, x):
        x = self.features(x)
        x = self.classifier(x)
        return x
vgg16 = VGG16(num_classes=num_classes).to(dv)
if os.path.exists("best_vgg16.pth"):
    vgg16.load_state_dict(torch.load("best_vgg16.pth"))
    print("Loaded best_vgg16.pth")
else:
    print("best_vgg16.pth not found, training from scratch")
criterion = nn.CrossEntropyLoss()

```

Loaded best_vgg16.pth

Training (VGG16)

```

In [ ]: learning_rate = 1e-3 # Higher LR for training from scratch with batch no
optimizer = torch.optim.SGD(vgg16.parameters(), lr=learning_rate, momentu

# Learning rate scheduler: reduce LR when validation loss plateaus
scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
    optimizer, mode='min', factor=0.7, patience=5,
    min_lr=1e-6
)

# Evaluate loaded model to establish baseline
vgg16.eval()
baseline_preds, baseline_true = [], []
with torch.no_grad():
    for images, labels in val_dataloader:
        images, labels = images.to(dv), labels.to(dv)
        outputs = vgg16(images)
        _, predicted = torch.max(outputs.data, 1)
        baseline_preds.extend(predicted.cpu().numpy())
        baseline_true.extend(labels.cpu().numpy())
best_val_accuracy = accuracy_score(baseline_true, baseline_preds)
print(f"Loaded model baseline validation accuracy: {best_val_accuracy:.4f}

num_epochs = 50
train_losses = []
val_losses = []
best_model_path = "best_vgg16.pth"

print("Starting VGG16 training with variable learning rate scheduling...")
for epoch in range(num_epochs):
    # Training phase
    vgg16.train()
    epoch_loss = 0.0

    for images, labels in train_dataloader:
        images, labels = images.to(dv), labels.to(dv)
        outputs = vgg16(images)
        loss = criterion(outputs, labels)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

```

```

        epoch_loss += loss.item()

    avg_train_loss = epoch_loss / len(train_dataloader)
    train_losses.append(avg_train_loss)

    # Validation phase
    vgg16.eval()
    val_epoch_loss = 0.0
    val_preds, val_true = [], []
    with torch.no_grad():
        for images, labels in val_dataloader:
            images, labels = images.to(dv), labels.to(dv)
            outputs = vgg16(images)
            loss = criterion(outputs, labels)
            val_epoch_loss += loss.item()

            _, predicted = torch.max(outputs.data, 1)
            val_preds.extend(predicted.cpu().numpy())
            val_true.extend(labels.cpu().numpy())

    avg_val_loss = val_epoch_loss / len(val_dataloader)
    val_losses.append(avg_val_loss)

    # Calculate validation accuracy
    val_accuracy = accuracy_score(val_true, val_preds)

    # Save model if validation accuracy improves
    if val_accuracy > best_val_accuracy:
        best_val_accuracy = val_accuracy
        torch.save(vgg16.state_dict(), best_model_path)
        print(f" → Model saved! (Val Accuracy: {val_accuracy:.4f})")

    # Learning rate scheduler step
    scheduler.step(avg_val_loss)

    if (epoch + 1) % 5 == 0:
        current_lr = optimizer.param_groups[0]['lr']
        print(f"Epoch [{epoch+1}/{num_epochs}], Train Loss: {avg_train_lo

print("Training complete!")
print(f"Final training loss: {train_losses[-1]:.4f}")
print(f"Final validation loss: {val_losses[-1]:.4f}")
print(f"Best validation accuracy: {best_val_accuracy:.4f}")

```

DenseNet121

```

In [83]: class Bottleneck(nn.Module):
    def __init__(self, in_channel, growth_rate):
        super(Bottleneck, self).__init__()

        self.bn1 = nn.BatchNorm2d(in_channel)
        self.conv1 = nn.Conv2d(in_channel, 4*growth_rate, kernel_size=1)

        self.bn2 = nn.BatchNorm2d(4*growth_rate)
        self.conv2 = nn.Conv2d(4*growth_rate, growth_rate, kernel_size=3,

    def forward(self, x):
        out = self.conv1(nn.functional.relu(self.bn1(x)))

```

```
        out = self.conv2(nn.functional.relu(self.bn2(out)))
        out = torch.cat([x,out], dim=1)
        return out

class DenseBlock(nn.Module):
    def __init__(self, num_bottlenecks, in_channel, growth_rate):
        super(DenseBlock, self).__init__()
        layers = []
        for i in range(num_bottlenecks):
            layers.append(Bottleneck(in_channel + i*growth_rate, growth_r
        self.block = nn.Sequential(*layers)

    def forward(self,x):
        return self.block(x)

class Transition(nn.Module):
    def __init__(self, in_channel, out_channel):
        super(Transition, self).__init__()
        self.bn = nn.BatchNorm2d(in_channel)
        self.conv = nn.Conv2d(in_channel, out_channel, kernel_size=1)
        self.pool = nn.AvgPool2d(kernel_size=2, stride=2)

    def forward(self,x):
        out = self.conv(nn.functional.relu(self.bn(x)))
        out = self.pool(out)
        return out

class DenseNet121(nn.Module):
    def __init__(self, num_blocks, growth_rate=32, num_classes=4):
        super(DenseNet121, self).__init__()
        self.growth_rate = growth_rate

        # Initial convolution
        self.conv1 = nn.Conv2d(3, 2*growth_rate, kernel_size=7, stride=2,
        self.bn1 = nn.BatchNorm2d(2*growth_rate)
        self.pool1 = nn.MaxPool2d(kernel_size=3, stride=2, padding=1)

        # Build dense blocks and transitions
        num_channels = 2 * growth_rate
        self.dense_blocks = nn.ModuleList()
        self.transitions = nn.ModuleList()

        for i, num_block in enumerate(num_blocks):
            self.dense_blocks.append(DenseBlock(num_block, num_channels,
            num_channels += num_block * growth_rate

            # Don't add transition after last dense block
            if i < len(num_blocks) - 1:
                self.transitions.append(Transition(num_channels, num_chan
                num_channels //= 2

        # Final layers
        self.bn_final = nn.BatchNorm2d(num_channels)
        self.pool_final = nn.AdaptiveAvgPool2d((1, 1))
        self.classifier = nn.Linear(num_channels, num_classes)

    def forward(self, x):
        # Initial convolution and pooling
        x = self.pool1(nn.functional.relu(self.bn1(self.conv1(x))))
```

```

        # Dense blocks and transitions
        for i, dense_block in enumerate(self.dense_blocks):
            x = dense_block(x)
            if i < len(self.transitions):
                x = self.transitions[i](x)

        # Final layers
        x = nn.functional.relu(self.bn_final(x))
        x = self.pool_final(x)
        x = x.view(x.size(0), -1)
        x = self.classifier(x)
        return x

densenet121 = DenseNet121(num_blocks=[6, 12, 24, 16], growth_rate=32, num
if os.path.exists("best_densenet121.pth"):
    densenet121.load_state_dict(torch.load("best_densenet121.pth"))
    print("Loaded best_densenet121.pth")
else:
    print("best_densenet121.pth not found, training from scratch")
criterion = nn.CrossEntropyLoss()

```

Loaded best_densenet121.pth

Training (DenseNet121)

```

In [ ]: # Train DenseNet121
learning_rate = 1e-3
optimizer = torch.optim.SGD(densenet121.parameters(), lr=learning_rate, m

scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
    optimizer, mode='min', factor=0.7, patience=5,
    min_lr=1e-6
)

# Evaluate loaded model to establish baseline
densenet121.eval()
baseline_preds, baseline_true = [], []
with torch.no_grad():
    for images, labels in val_dataloader:
        images, labels = images.to(dv), labels.to(dv)
        outputs = densenet121(images)
        _, predicted = torch.max(outputs.data, 1)
        baseline_preds.extend(predicted.cpu().numpy())
        baseline_true.extend(labels.cpu().numpy())
best_val_accuracy = accuracy_score(baseline_true, baseline_preds)
print(f"Loaded model baseline validation accuracy: {best_val_accuracy:.4f}

num_epochs = 50
train_losses = []
val_losses = []
best_model_path = "best_densenet121.pth"

print("Starting DenseNet121 training with variable learning rate scheduli
for epoch in range(num_epochs):
    # Training phase
    densenet121.train()
    epoch_loss = 0.0

    for images, labels in train_dataloader:
        images, labels = images.to(dv), labels.to(dv)

```

```

        outputs = densenet121(images)
        loss = criterion(outputs, labels)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        epoch_loss += loss.item()

    avg_train_loss = epoch_loss / len(train_dataloader)
    train_losses.append(avg_train_loss)

    # Validation phase
    densenet121.eval()
    val_epoch_loss = 0.0
    val_preds, val_true = [], []
    with torch.no_grad():
        for images, labels in val_dataloader:
            images, labels = images.to(dv), labels.to(dv)
            outputs = densenet121(images)
            loss = criterion(outputs, labels)
            val_epoch_loss += loss.item()

            _, predicted = torch.max(outputs.data, 1)
            val_preds.extend(predicted.cpu().numpy())
            val_true.extend(labels.cpu().numpy())

    avg_val_loss = val_epoch_loss / len(val_dataloader)
    val_losses.append(avg_val_loss)

    # Calculate validation accuracy
    val_accuracy = accuracy_score(val_true, val_preds)

    # Save model if validation accuracy improves
    if val_accuracy > best_val_accuracy:
        best_val_accuracy = val_accuracy
        torch.save(densenet121.state_dict(), best_model_path)
        print(f" → Model saved! (Val Accuracy: {val_accuracy:.4f})")

    # Learning rate scheduler step
    scheduler.step(avg_val_loss)

    if (epoch + 1) % 5 == 0:
        current_lr = optimizer.param_groups[0]['lr']
        print(f"Epoch [{epoch+1}/{num_epochs}], Train Loss: {avg_train_lo

print("Training complete!")
print(f"Final training loss: {train_losses[-1]:.4f}")
print(f"Final validation loss: {val_losses[-1]:.4f}")
print(f"Best validation accuracy: {best_val_accuracy:.4f}")

```

ResNet18

```

In [84]: class BasicBlock(nn.Module):
        """
        Basic residual block used in ResNet18/34.
        Two 3x3 convs with a skip connection. No bottleneck.
        expansion=1 because output channels == input channels (no expansion).

```

```

"""
expansion = 1

def __init__(self, in_channels, out_channels, stride=1, downsample=None):
    super(BasicBlock, self).__init__()

    # First 3x3 conv (stride applied here for spatial downsampling)
    self.conv1 = nn.Conv2d(in_channels, out_channels, kernel_size=3,
                             stride=stride, padding=1, bias=False)
    self.bn1 = nn.BatchNorm2d(out_channels)

    # Second 3x3 conv (always stride=1, no further downsampling)
    self.conv2 = nn.Conv2d(out_channels, out_channels, kernel_size=3,
                             stride=1, padding=1, bias=False)
    self.bn2 = nn.BatchNorm2d(out_channels)

    self.relu = nn.ReLU(inplace=True)
    self.downsample = downsample

def forward(self, x):
    identity = x

    out = self.relu(self.bn1(self.conv1(x)))
    out = self.bn2(self.conv2(out)) # No ReLU before skip add

    if self.downsample is not None:
        identity = self.downsample(x)

    out += identity # Skip connection
    out = self.relu(out)
    return out

class ResNet18(nn.Module):
    """
    ResNet18: 4 stages with [2, 2, 2, 2] basic blocks.
    Input: (B, 3, 224, 224) -> Output: (B, num_classes)
    """
    def __init__(self, num_classes=4):
        super(ResNet18, self).__init__()

        self.in_channels = 64

        # Stem: 7x7 conv + maxpool (224 -> 56)
        self.stem = nn.Sequential(
            nn.Conv2d(3, 64, kernel_size=7, stride=2, padding=3, bias=False),
            nn.BatchNorm2d(64),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=3, stride=2, padding=1)
        )

        # 4 residual stages - [2,2,2,2] blocks, channels double each stage
        self.layer1 = self._make_layer(out_channels=64, num_blocks=2, stride=1)
        self.layer2 = self._make_layer(out_channels=128, num_blocks=2, stride=2)
        self.layer3 = self._make_layer(out_channels=256, num_blocks=2, stride=2)
        self.layer4 = self._make_layer(out_channels=512, num_blocks=2, stride=2)

        # Head
        self.avgpool = nn.AdaptiveAvgPool2d((1, 1))
        # expansion=1 so final feature vector is 512, not 2048

```

```

        self.classifier = nn.Sequential(
            nn.Dropout(p=0.3),  # Light dropout: helps on small datasets
            nn.Linear(512 * BasicBlock.expansion, num_classes)
        )

    # Kaiming initialization
    for m in self.modules():
        if isinstance(m, nn.Conv2d):
            nn.init.kaiming_normal_(m.weight, mode='fan_out', nonline
        elif isinstance(m, nn.BatchNorm2d):
            nn.init.constant_(m.weight, 1)
            nn.init.constant_(m.bias, 0)

    def _make_layer(self, out_channels, num_blocks, stride):
        downsample = None

        if stride != 1 or self.in_channels != out_channels * BasicBlock.e
            downsample = nn.Sequential(
                nn.Conv2d(self.in_channels, out_channels * BasicBlock.exp
                    kernel_size=1, stride=stride, bias=False),
                nn.BatchNorm2d(out_channels * BasicBlock.expansion)
            )

        layers = [BasicBlock(self.in_channels, out_channels, stride, down
            self.in_channels = out_channels * BasicBlock.expansion

        for _ in range(1, num_blocks):
            layers.append(BasicBlock(self.in_channels, out_channels))

        return nn.Sequential(*layers)

    def forward(self, x):
        x = self.stem(x)
        x = self.layer1(x)
        x = self.layer2(x)
        x = self.layer3(x)
        x = self.layer4(x)
        x = self.avgpool(x)
        x = torch.flatten(x, 1)
        x = self.classifier(x)
        return x

resnet18 = ResNet18(num_classes=num_classes).to(dv)
if os.path.exists("best_resnet18.pth"):
    resnet18.load_state_dict(torch.load("best_resnet18.pth"))
    print("Loaded best_resnet18.pth")
else:
    print("best_resnet18.pth not found, training from scratch")
criterion = nn.CrossEntropyLoss()

```

Loaded best_resnet18.pth

Training (ResNet18)

```

In [ ]: learning_rate = 1e-2
optimizer = torch.optim.SGD(
    resnet18.parameters(),
    lr=learning_rate,

```

```
        momentum=0.9,
        weight_decay=1e-4
    )

    scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(
        optimizer, T_max=100, eta_min=1e-6
    )

    # Evaluate loaded model to establish baseline
    resnet18.eval()
    baseline_preds, baseline_true = [], []
    with torch.no_grad():
        for images, labels in val_dataloader:
            images, labels = images.to(dv), labels.to(dv)
            outputs = resnet18(images)
            _, predicted = torch.max(outputs.data, 1)
            baseline_preds.extend(predicted.cpu().numpy())
            baseline_true.extend(labels.cpu().numpy())
    best_val_accuracy = accuracy_score(baseline_true, baseline_preds)
    print(f"Loaded model baseline validation accuracy: {best_val_accuracy:.4f}")

    num_epochs = 100
    train_losses = []
    val_losses = []
    best_model_path = "best_resnet18.pth"

    print("Starting ResNet18 training...")
    for epoch in range(num_epochs):
        resnet18.train()
        epoch_loss = 0.0

        for images, labels in train_dataloader:
            images, labels = images.to(dv), labels.to(dv)
            outputs = resnet18(images)
            loss = criterion(outputs, labels)

            optimizer.zero_grad()
            loss.backward()
            torch.nn.utils.clip_grad_norm_(resnet18.parameters(), max_norm=1.0)
            optimizer.step()
            epoch_loss += loss.item()

        avg_train_loss = epoch_loss / len(train_dataloader)
        train_losses.append(avg_train_loss)

        resnet18.eval()
        val_epoch_loss = 0.0
        val_preds, val_true = [], []
        with torch.no_grad():
            for images, labels in val_dataloader:
                images, labels = images.to(dv), labels.to(dv)
                outputs = resnet18(images)
                loss = criterion(outputs, labels)
                val_epoch_loss += loss.item()

                _, predicted = torch.max(outputs.data, 1)
                val_preds.extend(predicted.cpu().numpy())
                val_true.extend(labels.cpu().numpy())

        avg_val_loss = val_epoch_loss / len(val_dataloader)
```

```

val_losses.append(avg_val_loss)

val_accuracy = accuracy_score(val_true, val_preds)

if val_accuracy > best_val_accuracy:
    best_val_accuracy = val_accuracy
    torch.save(resnet18.state_dict(), best_model_path)
    print(f" → Model saved! (Val Accuracy: {val_accuracy:.4f})")

scheduler.step()

if (epoch + 1) % 5 == 0:
    current_lr = optimizer.param_groups[0]['lr']
    print(f"Epoch [{epoch+1}/{num_epochs}], Train Loss: {avg_train_lo
          f"Val Loss: {avg_val_loss:.4f}, Val Acc: {val_accuracy:.4f}")

print("Training complete!")
print(f"Best validation accuracy: {best_val_accuracy:.4f}")

```

MobileNet

```

In [85]: # Inverted Residual Block
class InvertedResidual(nn.Module):
    def __init__(self, in_channels, out_channels, stride, expand_ratio):
        super(InvertedResidual, self).__init__()

        self.stride = stride
        hidden_dim = in_channels * expand_ratio

        self.use_residual = (stride == 1 and in_channels == out_channels)

        layers = []

        if expand_ratio != 1:
            layers.extend([
                nn.Conv2d(in_channels, hidden_dim, kernel_size=1, bias=False),
                nn.BatchNorm2d(hidden_dim),
                nn.ReLU6(inplace=True)
            ])

        layers.extend([
            nn.Conv2d(hidden_dim, hidden_dim, kernel_size=3, stride=stride,
                      padding=1, groups=hidden_dim, bias=False),
            nn.BatchNorm2d(hidden_dim),
            nn.ReLU6(inplace=True)
        ])

        layers.extend([
            nn.Conv2d(hidden_dim, out_channels, kernel_size=1, bias=False),
            nn.BatchNorm2d(out_channels)
            # No activation here – linear bottleneck by design
        ])

        self.block = nn.Sequential(*layers)

    def forward(self, x):
        if self.use_residual:
            return x + self.block(x)

```

```

        return self.block(x)

class MobileNetV2(nn.Module):
    def __init__(self, num_classes=4):
        super(MobileNetV2, self).__init__()

        self.conv1 = nn.Sequential(
            nn.Conv2d(3, 32, kernel_size=3, stride=2, padding=1, bias=False),
            nn.BatchNorm2d(32),
            nn.ReLU6(inplace=True)
        )

        # Reduced expansion factor (4 instead of 6) for small dataset:
        # less intermediate width means fewer parameters the small
        # dataset has to supervise in each depthwise conv
        config = [
            # t, c, n, s
            [1, 16, 1, 1],
            [4, 24, 2, 2], # expansion 6->4
            [4, 32, 3, 2], # expansion 6->4
            [4, 64, 4, 2], # expansion 6->4
            [4, 96, 3, 1], # expansion 6->4
            [4, 160, 3, 2], # expansion 6->4
            [4, 320, 1, 1], # expansion 6->4
        ]

        layers = []
        in_channels = 32

        for t, c, n, s in config:
            for i in range(n):
                stride = s if i == 0 else 1
                layers.append(InvertedResidual(in_channels, c, stride, ex
                in_channels = c

        self.blocks = nn.Sequential(*layers)

        self.conv2 = nn.Sequential(
            nn.Conv2d(in_channels, 1280, kernel_size=1, bias=False),
            nn.BatchNorm2d(1280),
            nn.ReLU6(inplace=True)
        )

        self.pool = nn.AdaptiveAvgPool2d((1, 1))

        # Added Dropout before classifier, consistent with ResNet18
        self.classifier = nn.Sequential(
            nn.Dropout(p=0.3),
            nn.Linear(1280, num_classes)
        )

        # Kaiming initialization, consistent with ResNet18 and InceptionV
        for m in self.modules():
            if isinstance(m, nn.Conv2d):
                nn.init.kaiming_normal_(m.weight, mode='fan_out', nonline
            elif isinstance(m, nn.BatchNorm2d):
                nn.init.constant_(m.weight, 1)
                nn.init.constant_(m.bias, 0)

```

```

    def forward(self, x):
        x = self.conv1(x)
        x = self.blocks(x)
        x = self.conv2(x)
        x = self.pool(x)
        x = torch.flatten(x, 1)
        x = self.classifier(x)
        return x

mobilenetv2 = MobileNetV2(num_classes=num_classes).to(dv)

if os.path.exists("best_mobilenetv2.pth"):
    mobilenetv2.load_state_dict(torch.load("best_mobilenetv2.pth"))
    print("Loaded best_mobilenetv2.pth")
else:
    print("best_mobilenetv2.pth not found, training from scratch")

criterion = nn.CrossEntropyLoss()

```

Loaded best_mobilenetv2.pth

Training (MobileNetV2)

```

In [46]: learning_rate = 1e-3    # Lower start than ResNet/Inception – MobileNet's
                                     # depthwise convs are more sensitive to large LR s

optimizer = torch.optim.SGD(
    mobilenetv2.parameters(),
    lr=learning_rate,
    momentum=0.9,
    weight_decay=1e-4
)

# Warmup for 5 epochs then cosine decay.
# The warmup lets depthwise and pointwise convs begin co-adapting
# at a safe LR before the main training phase kicks in.
def warmup_cosine_schedule(epoch, warmup_epochs=5, T_max=100):
    if epoch < warmup_epochs:
        return (epoch + 1) / warmup_epochs    # Linear warmup: 0.2 -> 1.0
    progress = (epoch - warmup_epochs) / (T_max - warmup_epochs)
    return 0.5 * (1 + np.cos(np.pi * progress))    # Cosine decay after war

scheduler = torch.optim.lr_scheduler.LambdaLR(
    optimizer,
    lr_lambda=warmup_cosine_schedule
)

mobilenetv2.eval()
baseline_preds, baseline_true = [], []
with torch.no_grad():
    for images, labels in val_dataloader:
        images, labels = images.to(dv), labels.to(dv)
        outputs = mobilenetv2(images)
        _, predicted = torch.max(outputs.data, 1)
        baseline_preds.extend(predicted.cpu().numpy())
        baseline_true.extend(labels.cpu().numpy())
best_val_accuracy = accuracy_score(baseline_true, baseline_preds)
print(f"Loaded model baseline validation accuracy: {best_val_accuracy:.4f}

```

```
num_epochs = 100
train_losses = []
val_losses = []
best_model_path = "best_mobilenetv2.pth"

print("Starting MobileNetV2 training...")
for epoch in range(num_epochs):
    mobilenetv2.train()
    epoch_loss = 0.0
    train_preds, train_true = [], []

    for images, labels in train_dataloader:
        images, labels = images.to(dv), labels.to(dv)
        outputs = mobilenetv2(images)
        loss = criterion(outputs, labels)

        optimizer.zero_grad()
        loss.backward()
        #torch.nn.utils.clip_grad_norm_(mobilenetv2.parameters(), max_norm)
        optimizer.step()
        epoch_loss += loss.item()

        _, predicted = torch.max(outputs.data, 1)
        train_preds.extend(predicted.cpu().numpy())
        train_true.extend(labels.cpu().numpy())

    avg_train_loss = epoch_loss / len(train_dataloader)
    train_losses.append(avg_train_loss)
    train_accuracy = accuracy_score(train_true, train_preds)

    mobilenetv2.eval()
    val_epoch_loss = 0.0
    val_preds, val_true = [], []
    with torch.no_grad():
        for images, labels in val_dataloader:
            images, labels = images.to(dv), labels.to(dv)
            outputs = mobilenetv2(images)
            loss = criterion(outputs, labels)
            val_epoch_loss += loss.item()
            _, predicted = torch.max(outputs.data, 1)
            val_preds.extend(predicted.cpu().numpy())
            val_true.extend(labels.cpu().numpy())

    avg_val_loss = val_epoch_loss / len(val_dataloader)
    val_losses.append(avg_val_loss)
    val_accuracy = accuracy_score(val_true, val_preds)

    if val_accuracy > best_val_accuracy:
        best_val_accuracy = val_accuracy
        torch.save(mobilenetv2.state_dict(), best_model_path)
        print(f" → Model saved! (Val Accuracy: {val_accuracy:.4f})")

    scheduler.step()

    if (epoch + 1) % 5 == 0:
        current_lr = optimizer.param_groups[0]['lr']
        print(f"Epoch [{epoch+1}/{num_epochs}], Train Loss: {avg_train_loss:.4f}, Val Loss: {avg_val_loss:.4f}, Val Acc: {val_accuracy:.4f}")
```

```
print("Training complete!")
print(f"Best validation accuracy: {best_val_accuracy:.4f}")
```

Loaded model baseline validation accuracy: 0.6733

Starting MobileNetV2 training...

```
Epoch [5/100], Train Loss: 0.3844, Train Acc: 0.8514, Val Loss: 1.7499, Val Acc: 0.5867, LR: 0.001000
Epoch [10/100], Train Loss: 0.4351, Train Acc: 0.8443, Val Loss: 1.6313, Val Acc: 0.5733, LR: 0.000993
Epoch [15/100], Train Loss: 0.4260, Train Acc: 0.8486, Val Loss: 1.2401, Val Acc: 0.6067, LR: 0.000973
Epoch [20/100], Train Loss: 0.4642, Train Acc: 0.8329, Val Loss: 1.2214, Val Acc: 0.5867, LR: 0.000940
Epoch [25/100], Train Loss: 0.4339, Train Acc: 0.8443, Val Loss: 1.4355, Val Acc: 0.5933, LR: 0.000895
  → Model saved! (Val Accuracy: 0.6800)
Epoch [30/100], Train Loss: 0.2710, Train Acc: 0.9057, Val Loss: 1.2838, Val Acc: 0.6333, LR: 0.000839
  → Model saved! (Val Accuracy: 0.6867)
Epoch [35/100], Train Loss: 0.2313, Train Acc: 0.9214, Val Loss: 1.1230, Val Acc: 0.6867, LR: 0.000773
  → Model saved! (Val Accuracy: 0.6933)
Epoch [40/100], Train Loss: 0.2865, Train Acc: 0.8900, Val Loss: 1.1434, Val Acc: 0.6467, LR: 0.000701
Epoch [45/100], Train Loss: 0.2253, Train Acc: 0.9229, Val Loss: 1.1615, Val Acc: 0.6600, LR: 0.000623
Epoch [50/100], Train Loss: 0.2349, Train Acc: 0.9286, Val Loss: 1.0820, Val Acc: 0.6800, LR: 0.000541
Epoch [55/100], Train Loss: 0.2116, Train Acc: 0.9314, Val Loss: 1.0395, Val Acc: 0.6933, LR: 0.000459
  → Model saved! (Val Accuracy: 0.7067)
  → Model saved! (Val Accuracy: 0.7200)
Epoch [60/100], Train Loss: 0.1660, Train Acc: 0.9471, Val Loss: 1.0101, Val Acc: 0.7200, LR: 0.000377
Epoch [65/100], Train Loss: 0.1786, Train Acc: 0.9357, Val Loss: 1.0542, Val Acc: 0.6933, LR: 0.000299
Epoch [70/100], Train Loss: 0.1687, Train Acc: 0.9457, Val Loss: 1.0489, Val Acc: 0.6733, LR: 0.000227
Epoch [75/100], Train Loss: 0.1328, Train Acc: 0.9643, Val Loss: 1.0306, Val Acc: 0.7133, LR: 0.000161
  → Model saved! (Val Accuracy: 0.7333)
Epoch [80/100], Train Loss: 0.1019, Train Acc: 0.9714, Val Loss: 1.0643, Val Acc: 0.6867, LR: 0.000105
Epoch [85/100], Train Loss: 0.1026, Train Acc: 0.9657, Val Loss: 1.0223, Val Acc: 0.7067, LR: 0.000060
Epoch [90/100], Train Loss: 0.0953, Train Acc: 0.9686, Val Loss: 1.0469, Val Acc: 0.6867, LR: 0.000027
Epoch [95/100], Train Loss: 0.1183, Train Acc: 0.9600, Val Loss: 1.0721, Val Acc: 0.6933, LR: 0.000007
Epoch [100/100], Train Loss: 0.1354, Train Acc: 0.9557, Val Loss: 1.0472, Val Acc: 0.6867, LR: 0.000000
Training complete!
Best validation accuracy: 0.7333
```

InceptionV3

```
In [86]: class BasicConv2d(nn.Module):
```

```

"""
Reusable Conv -> BN -> ReLU building block used throughout Inception
Almost every convolution in the network uses this exact pattern,
so factoring it out keeps the code clean and consistent.
"""
def __init__(self, in_channels, out_channels, **kwargs):
    super(BasicConv2d, self).__init__()
    # bias=False because BN already handles the bias term
    self.conv = nn.Conv2d(in_channels, out_channels, bias=False, **kw
    self.bn = nn.BatchNorm2d(out_channels)
    self.relu = nn.ReLU(inplace=True)

def forward(self, x):
    return self.relu(self.bn(self.conv(x)))

class InceptionA(nn.Module):
    """
    Inception module used in the early stages of the network.
    Four parallel branches are concatenated along the channel dimension:
    - Branch 1: single 1x1 conv
    - Branch 2: 1x1 conv -> 5x5 conv
    - Branch 3: 1x1 conv -> two 3x3 convs (cheaper than one 5x5)
    - Branch 4: avg pool -> 1x1 conv
    The pool_features parameter controls branch 4 output size and varies
    across the three InceptionA modules in the network.
    """
    def __init__(self, in_channels, pool_features):
        super(InceptionA, self).__init__()

        # Branch 1: simple 1x1
        self.branch1 = BasicConv2d(in_channels, 64, kernel_size=1)

        # Branch 2: 1x1 -> 5x5
        self.branch2 = nn.Sequential(
            BasicConv2d(in_channels, 48, kernel_size=1),
            BasicConv2d(48, 64, kernel_size=5, padding=2)
        )

        # Branch 3: 1x1 -> 3x3 -> 3x3 (factorized 5x5)
        self.branch3 = nn.Sequential(
            BasicConv2d(in_channels, 64, kernel_size=1),
            BasicConv2d(64, 96, kernel_size=3, padding=1),
            BasicConv2d(96, 96, kernel_size=3, padding=1)
        )

        # Branch 4: avg pool -> 1x1
        self.branch4 = nn.Sequential(
            nn.AvgPool2d(kernel_size=3, stride=1, padding=1),
            BasicConv2d(in_channels, pool_features, kernel_size=1)
        )

    def forward(self, x):
        b1 = self.branch1(x)
        b2 = self.branch2(x)
        b3 = self.branch3(x)
        b4 = self.branch4(x)
        # Concatenate all branches along channel dimension
        return torch.cat([b1, b2, b3, b4], dim=1)

```

```

class InceptionB(nn.Module):
    """
    Grid reduction module: reduces spatial dimensions from 35x35 -> 17x17
    Two branches (no pooling branch), then a max pool branch:
    - Branch 1: 1x1 -> 3x3 with stride=2 (spatial downsampling)
    - Branch 2: 1x1 -> 3x3 -> 3x3 with stride=2
    - Branch 3: max pool stride=2
    Stride=2 in branches 1 and 2 (instead of a separate pooling layer)
    is how Inception reduces spatial size without losing information.
    """
    def __init__(self, in_channels):
        super(InceptionB, self).__init__()

        self.branch1 = BasicConv2d(in_channels, 384, kernel_size=3, stride=2)

        self.branch2 = nn.Sequential(
            BasicConv2d(in_channels, 64, kernel_size=1),
            BasicConv2d(64, 96, kernel_size=3, padding=1),
            BasicConv2d(96, 96, kernel_size=3, stride=2)
        )

        self.branch3 = nn.MaxPool2d(kernel_size=3, stride=2)

    def forward(self, x):
        b1 = self.branch1(x)
        b2 = self.branch2(x)
        b3 = self.branch3(x)
        return torch.cat([b1, b2, b3], dim=1)

class InceptionC(nn.Module):
    """
    Factorized convolution module used in the middle stages (17x17 grid).
    The key idea: an nxn convolution is factorized into a 1xn followed by
    an nx1 convolution. This gives the same receptive field at lower cost
    For example, one 7x7 = 49 multiplications, but 1x7 + 7x1 = 14.
    channels_7x7 controls the intermediate channel count and increases
    across the four InceptionC modules (128 -> 160 -> 160 -> 192).
    """
    def __init__(self, in_channels, channels_7x7):
        super(InceptionC, self).__init__()
        c7 = channels_7x7

        # Branch 1: 1x1
        self.branch1 = BasicConv2d(in_channels, 192, kernel_size=1)

        # Branch 2: 1x1 -> 1x7 -> 7x1
        self.branch2 = nn.Sequential(
            BasicConv2d(in_channels, c7, kernel_size=1),
            BasicConv2d(c7, c7, kernel_size=(1, 7), padding=(0, 3)),
            BasicConv2d(c7, 192, kernel_size=(7, 1), padding=(3, 0))
        )

        # Branch 3: 1x1 -> 7x1 -> 1x7 -> 7x1 -> 1x7 (deeper factorization)
        self.branch3 = nn.Sequential(
            BasicConv2d(in_channels, c7, kernel_size=1),
            BasicConv2d(c7, c7, kernel_size=(7, 1), padding=(3, 0)),
            BasicConv2d(c7, c7, kernel_size=(1, 7), padding=(0, 3)),
            BasicConv2d(c7, c7, kernel_size=(7, 1), padding=(3, 0))
        )

```

```
        BasicConv2d(c7, 192, kernel_size=(1, 7), padding=(0,
    )

    # Branch 4: avg pool -> 1x1
    self.branch4 = nn.Sequential(
        nn.AvgPool2d(kernel_size=3, stride=1, padding=1),
        BasicConv2d(in_channels, 192, kernel_size=1)
    )

    def forward(self, x):
        b1 = self.branch1(x)
        b2 = self.branch2(x)
        b3 = self.branch3(x)
        b4 = self.branch4(x)
        return torch.cat([b1, b2, b3, b4], dim=1)

class InceptionD(nn.Module):
    """
    Second grid reduction module: 17x17 -> 8x8.
    Same philosophy as InceptionB – uses strided convolutions
    across branches rather than a separate downsampling layer.
    """
    def __init__(self, in_channels):
        super(InceptionD, self).__init__()

        self.branch1 = nn.Sequential(
            BasicConv2d(in_channels, 192, kernel_size=1),
            BasicConv2d(192, 320, kernel_size=3, stride=2)
        )

        self.branch2 = nn.Sequential(
            BasicConv2d(in_channels, 192, kernel_size=1),
            BasicConv2d(192, 192, kernel_size=(1, 7), padding=(0,
            BasicConv2d(192, 192, kernel_size=(7, 1), padding=(3,
            BasicConv2d(192, 192, kernel_size=3, stride=2)
        )

        self.branch3 = nn.MaxPool2d(kernel_size=3, stride=2)

    def forward(self, x):
        b1 = self.branch1(x)
        b2 = self.branch2(x)
        b3 = self.branch3(x)
        return torch.cat([b1, b2, b3], dim=1)

class InceptionE(nn.Module):
    """
    Expanded filter bank module used in the final stages (8x8 grid).
    Branch 2 and 3 each split into two parallel 1x3 and 3x1 convolutions
    whose outputs are concatenated. This maximizes the variety of feature
    detectors at the coarsest spatial scale before classification.
    """
    def __init__(self, in_channels):
        super(InceptionE, self).__init__()

        # Branch 1: 1x1
        self.branch1 = BasicConv2d(in_channels, 320, kernel_size=1)
```

```

# Branch 2: 1x1 -> split into 1x3 and 3x1 in parallel
self.branch2_reduce = BasicConv2d(in_channels, 384, kernel_size=1)
self.branch2a = BasicConv2d(384, 384, kernel_size=(1, 3), padding=1)
self.branch2b = BasicConv2d(384, 384, kernel_size=(3, 1), padding=1)

# Branch 3: 1x1 -> 3x3 -> split into 1x3 and 3x1 in parallel
self.branch3_reduce = BasicConv2d(in_channels, 448, kernel_size=1)
self.branch3_conv = BasicConv2d(448, 384, kernel_size=3, padding=1)
self.branch3a = BasicConv2d(384, 384, kernel_size=(1, 3), padding=1)
self.branch3b = BasicConv2d(384, 384, kernel_size=(3, 1), padding=1)

# Branch 4: avg pool -> 1x1
self.branch4 = nn.Sequential(
    nn.AvgPool2d(kernel_size=3, stride=1, padding=1),
    BasicConv2d(in_channels, 192, kernel_size=1)
)

def forward(self, x):
    b1 = self.branch1(x)

    b2 = self.branch2_reduce(x)
    b2 = torch.cat([self.branch2a(b2), self.branch2b(b2)], dim=1) #

    b3 = self.branch3_conv(self.branch3_reduce(x))
    b3 = torch.cat([self.branch3a(b3), self.branch3b(b3)], dim=1) #

    b4 = self.branch4(x)

    return torch.cat([b1, b2, b3, b4], dim=1) # 320+768+768+192 = 20

class InceptionV3(nn.Module):
    """
    Inception V3: full architecture as described in
    'Rethinking the Inception Architecture for Computer Vision' (Szegedy)

    Spatial flow:
    Input 224x224 -> Stem -> 26x26 -> InceptionA x3 -> InceptionB
    -> 12x12 -> InceptionC x4 -> InceptionD -> 5x5 -> InceptionE x2
    -> AvgPool -> classifier

    Note: the original paper uses 299x299 input. We use 224x224 to stay
    consistent with the rest of the codebase. The spatial dimensions above
    reflect 224x224 input accordingly.
    """
    def __init__(self, num_classes=4):
        super(InceptionV3, self).__init__()

        # --- Stem ---
        # Aggressively reduces 224x224 down before the Inception modules
        # Uses strided convs rather than pooling to preserve more information
        self.stem = nn.Sequential(
            BasicConv2d(3, 32, kernel_size=3, stride=2), # 2
            BasicConv2d(32, 32, kernel_size=3), #
            BasicConv2d(32, 64, kernel_size=3, padding=1), #
            nn.MaxPool2d(kernel_size=3, stride=2), #
            BasicConv2d(64, 80, kernel_size=1), #
            BasicConv2d(80, 192, kernel_size=3), #
            nn.MaxPool2d(kernel_size=3, stride=2) #
        )

```

```

# --- InceptionA x3 (on 26x26 grid) ---
# pool_features varies: controls how many channels the avg pool
# branch contributes; increases to maintain total output channels
# as the network deepens (256 -> 288 -> 288)
self.inceptionA1 = InceptionA(192, pool_features=32) # out: 256
self.inceptionA2 = InceptionA(256, pool_features=64) # out: 288
self.inceptionA3 = InceptionA(288, pool_features=64) # out: 288

# --- InceptionB: 26x26 -> 12x12 ---
self.inceptionB = InceptionB(288) # out: 76

# --- InceptionC x4 (on 12x12 grid) ---
# channels_7x7 increases from 128 to 192 across the four modules,
# giving the factorized branches progressively more capacity
self.inceptionC1 = InceptionC(768, channels_7x7=128) # out: 768
self.inceptionC2 = InceptionC(768, channels_7x7=160) # out: 768
self.inceptionC3 = InceptionC(768, channels_7x7=160) # out: 768
self.inceptionC4 = InceptionC(768, channels_7x7=192) # out: 768

# --- InceptionD: 12x12 -> 5x5 ---
self.inceptionD = InceptionD(768) # out: 12

# --- InceptionE x2 (on 5x5 grid) ---
self.inceptionE1 = InceptionE(1280) # out: 20
self.inceptionE2 = InceptionE(2048) # out: 20

# --- Head ---
self.avgpool = nn.AdaptiveAvgPool2d((1, 1))
self.dropout = nn.Dropout(p=0.4) # Inception V3 uses dropout
self.classifier = nn.Linear(2048, num_classes)

# Kaiming initialization for conv layers, same as ResNet
for m in self.modules():
    if isinstance(m, nn.Conv2d):
        nn.init.kaiming_normal_(m.weight, mode='fan_out', nonline
    elif isinstance(m, nn.BatchNorm2d):
        nn.init.constant_(m.weight, 1)
        nn.init.constant_(m.bias, 0)

def forward(self, x):
    x = self.stem(x)

    x = self.inceptionA1(x)
    x = self.inceptionA2(x)
    x = self.inceptionA3(x)

    x = self.inceptionB(x)

    x = self.inceptionC1(x)
    x = self.inceptionC2(x)
    x = self.inceptionC3(x)
    x = self.inceptionC4(x)

    x = self.inceptionD(x)

    x = self.inceptionE1(x)
    x = self.inceptionE2(x)

    x = self.avgpool(x)

```

```

        x = torch.flatten(x, 1)
        x = self.dropout(x)
        x = self.classifier(x)
        return x

inceptionv3 = InceptionV3(num_classes=num_classes).to(dv)
if os.path.exists("best_inceptionv3.pth"):
    inceptionv3.load_state_dict(torch.load("best_inceptionv3.pth"))
    print("Loaded best_inceptionv3.pth")
else:
    print("best_inceptionv3.pth not found, training from scratch")
criterion = nn.CrossEntropyLoss()

```

Loaded best_inceptionv3.pth

Training (InceptionV3)

```

In [75]: learning_rate = 1e-2
optimizer = torch.optim.SGD(
    inceptionv3.parameters(),
    lr=learning_rate,
    momentum=0.9,
    weight_decay=1e-4
)

scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(
    optimizer, T_max=100, eta_min=1e-6
)

# Evaluate loaded model to establish baseline
inceptionv3.eval()
baseline_preds, baseline_true = [], []
with torch.no_grad():
    for images, labels in val_dataloader:
        images, labels = images.to(dv), labels.to(dv)
        outputs = inceptionv3(images)
        _, predicted = torch.max(outputs.data, 1)
        baseline_preds.extend(predicted.cpu().numpy())
        baseline_true.extend(labels.cpu().numpy())
best_val_accuracy = accuracy_score(baseline_true, baseline_preds)
print(f"Loaded model baseline validation accuracy: {best_val_accuracy:.4f}")

num_epochs = 100
train_losses = []
val_losses = []
best_model_path = "best_inceptionv3.pth"

print("Starting Inception V3 training...")
for epoch in range(num_epochs):
    inceptionv3.train()
    epoch_loss = 0.0

    for images, labels in train_dataloader:
        images, labels = images.to(dv), labels.to(dv)
        outputs = inceptionv3(images)
        loss = criterion(outputs, labels)

        optimizer.zero_grad()

```

```
        loss.backward()
        #torch.nn.utils.clip_grad_norm_(inceptionv3.parameters(), max_norm)
        optimizer.step()
        epoch_loss += loss.item()

    avg_train_loss = epoch_loss / len(train_dataloader)
    train_losses.append(avg_train_loss)

    inceptionv3.eval()
    val_epoch_loss = 0.0
    val_preds, val_true = [], []
    with torch.no_grad():
        for images, labels in val_dataloader:
            images, labels = images.to(dv), labels.to(dv)
            outputs = inceptionv3(images)
            loss = criterion(outputs, labels)
            val_epoch_loss += loss.item()

            _, predicted = torch.max(outputs.data, 1)
            val_preds.extend(predicted.cpu().numpy())
            val_true.extend(labels.cpu().numpy())

    avg_val_loss = val_epoch_loss / len(val_dataloader)
    val_losses.append(avg_val_loss)

    val_accuracy = accuracy_score(val_true, val_preds)

    if val_accuracy > best_val_accuracy:
        best_val_accuracy = val_accuracy
        torch.save(inceptionv3.state_dict(), best_model_path)
        print(f" → Model saved! (Val Accuracy: {val_accuracy:.4f})")

    scheduler.step()

    if (epoch + 1) % 5 == 0:
        current_lr = optimizer.param_groups[0]['lr']
        print(f"Epoch [{epoch+1}/{num_epochs}], Train Loss: {avg_train_loss:.4f}, Val Loss: {avg_val_loss:.4f}, Val Acc: {val_accuracy:.4f}")

print("Training complete!")
print(f"Best validation accuracy: {best_val_accuracy:.4f}")
```

Loaded model baseline validation accuracy: 0.3400

Starting Inception V3 training...

→ Model saved! (Val Accuracy: 0.4133)

Epoch [5/100], Train Loss: 1.3936, Train Acc: 0.9557, Val Loss: 1.4252, Val Acc: 0.3067, LR: 0.000994

Epoch [10/100], Train Loss: 1.3853, Train Acc: 0.9557, Val Loss: 1.4967, Val Acc: 0.2800, LR: 0.000976

Epoch [15/100], Train Loss: 1.3940, Train Acc: 0.9557, Val Loss: 1.4056, Val Acc: 0.3133, LR: 0.000946

Epoch [20/100], Train Loss: 1.3714, Train Acc: 0.9557, Val Loss: 1.4539, Val Acc: 0.3667, LR: 0.000905

Epoch [25/100], Train Loss: 1.3461, Train Acc: 0.9557, Val Loss: 1.3990, Val Acc: 0.3200, LR: 0.000854

Epoch [30/100], Train Loss: 1.3379, Train Acc: 0.9557, Val Loss: 1.4119, Val Acc: 0.3333, LR: 0.000794

Epoch [35/100], Train Loss: 1.2956, Train Acc: 0.9557, Val Loss: 1.4474, Val Acc: 0.3067, LR: 0.000727

Epoch [40/100], Train Loss: 1.2824, Train Acc: 0.9557, Val Loss: 1.3982, Val Acc: 0.3933, LR: 0.000655

Epoch [45/100], Train Loss: 1.2892, Train Acc: 0.9557, Val Loss: 1.4166, Val Acc: 0.3333, LR: 0.000579

Epoch [50/100], Train Loss: 1.2776, Train Acc: 0.9557, Val Loss: 1.4935, Val Acc: 0.3733, LR: 0.000501

Epoch [55/100], Train Loss: 1.2395, Train Acc: 0.9557, Val Loss: 1.4324, Val Acc: 0.3400, LR: 0.000422

Epoch [60/100], Train Loss: 1.2104, Train Acc: 0.9557, Val Loss: 1.3860, Val Acc: 0.3667, LR: 0.000346

Epoch [65/100], Train Loss: 1.1904, Train Acc: 0.9557, Val Loss: 1.3853, Val Acc: 0.4000, LR: 0.000274

Epoch [70/100], Train Loss: 1.2124, Train Acc: 0.9557, Val Loss: 1.3751, Val Acc: 0.3800, LR: 0.000207

→ Model saved! (Val Accuracy: 0.4200)

→ Model saved! (Val Accuracy: 0.4267)

→ Model saved! (Val Accuracy: 0.4333)

Epoch [75/100], Train Loss: 1.2044, Train Acc: 0.9557, Val Loss: 1.3987, Val Acc: 0.4333, LR: 0.000147

Epoch [80/100], Train Loss: 1.1583, Train Acc: 0.9557, Val Loss: 1.3837, Val Acc: 0.4133, LR: 0.000096

Epoch [85/100], Train Loss: 1.1520, Train Acc: 0.9557, Val Loss: 1.3932, Val Acc: 0.4333, LR: 0.000055

Epoch [90/100], Train Loss: 1.1526, Train Acc: 0.9557, Val Loss: 1.3955, Val Acc: 0.3933, LR: 0.000025

Epoch [95/100], Train Loss: 1.1321, Train Acc: 0.9557, Val Loss: 1.3852, Val Acc: 0.4333, LR: 0.000007

Epoch [100/100], Train Loss: 1.1723, Train Acc: 0.9557, Val Loss: 1.3837, Val Acc: 0.4133, LR: 0.000001

Training complete!

Best validation accuracy: 0.4333

Evaluation (All Models)

```
In [87]: # Network-agnostic evaluation function
def evaluate_model(model, model_name, val_loader, test_loader, criterion,
    """
    Evaluate a model on validation and test datasets.

    Args:
```

```

        model: PyTorch model to evaluate
        model_name: Name of the model for display
        val_loader: Validation DataLoader
        test_loader: Test DataLoader
        criterion: Loss function
        device: Device to run on (cpu/cuda)
        class_names: List of class names
    """
    model.eval()
    val_preds, val_true = [], []
    test_preds, test_true = [], []
    val_loss, test_loss = 0.0, 0.0

    # Validation evaluation
    with torch.no_grad():
        for images, labels in val_loader:
            images, labels = images.to(device), labels.to(device)
            outputs = model(images)
            loss = criterion(outputs, labels)
            val_loss += loss.item()

            _, predicted = torch.max(outputs.data, 1)
            val_preds.extend(predicted.cpu().numpy())
            val_true.extend(labels.cpu().numpy())

    val_accuracy = accuracy_score(val_true, val_preds)
    val_precision = precision_score(val_true, val_preds, average='weighted')
    val_recall = recall_score(val_true, val_preds, average='weighted', zero_divis
    val_f1 = f1_score(val_true, val_preds, average='weighted', zero_divis
    val_loss /= len(val_loader)

    # Test evaluation
    with torch.no_grad():
        for images, labels in test_loader:
            images, labels = images.to(device), labels.to(device)
            outputs = model(images)
            loss = criterion(outputs, labels)
            test_loss += loss.item()

            _, predicted = torch.max(outputs.data, 1)
            test_preds.extend(predicted.cpu().numpy())
            test_true.extend(labels.cpu().numpy())

    test_accuracy = accuracy_score(test_true, test_preds)
    test_precision = precision_score(test_true, test_preds, average='weig
    test_recall = recall_score(test_true, test_preds, average='weighted',
    test_f1 = f1_score(test_true, test_preds, average='weighted', zero_di
    test_loss /= len(test_loader)

    # Print Results
    print("\n" + "="*60)
    print(f"{model_name.upper()} EVALUATION RESULTS")
    print("="*60)

    print(f"\nVALIDATION:")
    print(f" Accuracy: {val_accuracy:.4f}")
    print(f" Precision: {val_precision:.4f}")
    print(f" Recall: {val_recall:.4f}")
    print(f" F1-Score: {val_f1:.4f}")
    print(f" Loss: {val_loss:.4f}")

```

```

print(f"\nTEST:")
print(f"  Accuracy:  {test_accuracy:.4f}")
print(f"  Precision:  {test_precision:.4f}")
print(f"  Recall:     {test_recall:.4f}")
print(f"  F1-Score:   {test_f1:.4f}")
print(f"  Loss:       {test_loss:.4f}")

print(f"\nCONFUSION MATRICES (TEXT):")

val_cm = confusion_matrix(val_true, val_preds)
test_cm = confusion_matrix(test_true, test_preds)

print(f"\nCONFUSION MATRICES (HEATMAPS):")

# Create figure with two subplots for confusion matrices
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Validation confusion matrix
sns.heatmap(val_cm, annot=True, fmt='d', cmap='Blues', ax=axes[0],
            xticklabels=class_names, yticklabels=class_names, cbar_kw
axes[0].set_title(f'{model_name} - Validation Confusion Matrix')
axes[0].set_ylabel('True Label')
axes[0].set_xlabel('Predicted Label')

# Test confusion matrix
sns.heatmap(test_cm, annot=True, fmt='d', cmap='Greens', ax=axes[1],
            xticklabels=class_names, yticklabels=class_names, cbar_kw
axes[1].set_title(f'{model_name} - Test Confusion Matrix')
axes[1].set_ylabel('True Label')
axes[1].set_xlabel('Predicted Label')

plt.tight_layout()
plt.show()

return {
    'model_name': model_name,
    'val_accuracy': val_accuracy,
    'val_precision': val_precision,
    'val_recall': val_recall,
    'val_f1': val_f1,
    'val_loss': val_loss,
    'test_accuracy': test_accuracy,
    'test_precision': test_precision,
    'test_recall': test_recall,
    'test_f1': test_f1,
    'test_loss': test_loss
}

vgg16_results = densenet_results = resnet18_results = mobilenetv2_results

# Evaluate the models
if os.path.exists("best_vgg16.pth"):
    vgg16.load_state_dict(torch.load("best_vgg16.pth"))
    vgg16_results = evaluate_model(vgg16, "VGG16", val_dataloader, test_d
if os.path.exists("best_densenet121.pth"):
    densenet121.load_state_dict(torch.load("best_densenet121.pth"))
    densenet_results = evaluate_model(densenet121, "DenseNet121", val_dat
if os.path.exists("best_resnet18.pth"):
    resnet18.load_state_dict(torch.load("best_resnet18.pth"))

```

```
resnet18_results = evaluate_model(resnet18, "ResNet18", val_data_loader)
if os.path.exists("best_mobilenetv2.pth"):
    mobilenetv2.load_state_dict(torch.load("best_mobilenetv2.pth"))
    mobilenetv2_results = evaluate_model(mobilenetv2, "MobileNetV2", val_data_loader)
if os.path.exists("best_inceptionv3.pth"):
    inceptionv3.load_state_dict(torch.load("best_inceptionv3.pth"))
    inceptionv3_results = evaluate_model(inceptionv3, "InceptionV3", val_data_loader)
```

VGG16 EVALUATION RESULTS

VALIDATION:

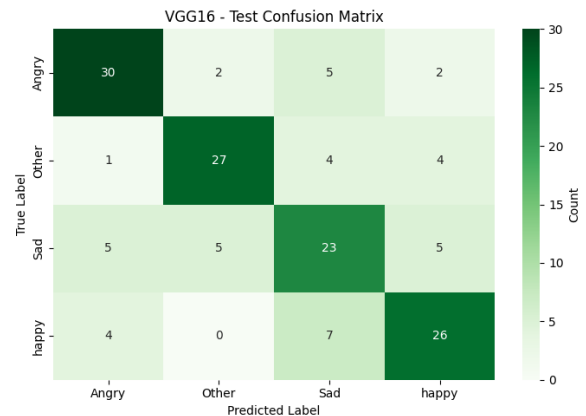
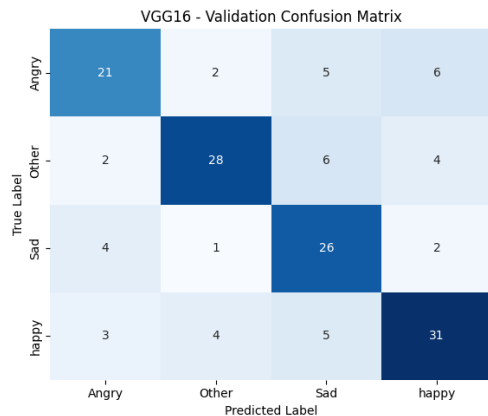
Accuracy: 0.7067
Precision: 0.7149
Recall: 0.7067
F1-Score: 0.7071
Loss: 1.0634

TEST:

Accuracy: 0.7067
Precision: 0.7083
Recall: 0.7067
F1-Score: 0.7073
Loss: 1.0608

CONFUSION MATRICES (TEXT):

CONFUSION MATRICES (HEATMAPS):



DENSENET121 EVALUATION RESULTS

VALIDATION:

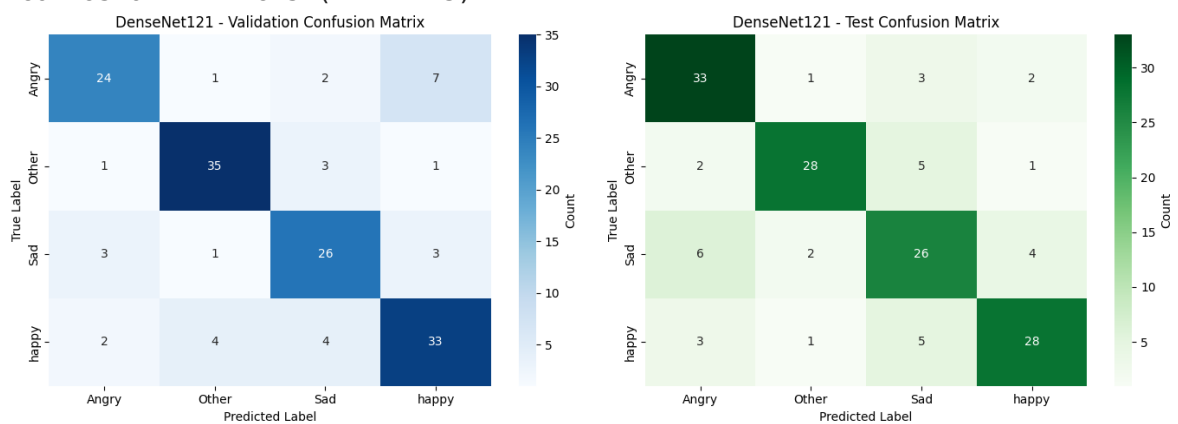
Accuracy: 0.7867
Precision: 0.7874
Recall: 0.7867
F1-Score: 0.7862
Loss: 0.7543

TEST:

Accuracy: 0.7667
Precision: 0.7712
Recall: 0.7667
F1-Score: 0.7673
Loss: 0.7506

CONFUSION MATRICES (TEXT):

CONFUSION MATRICES (HEATMAPS):



RESNET18 EVALUATION RESULTS

VALIDATION:

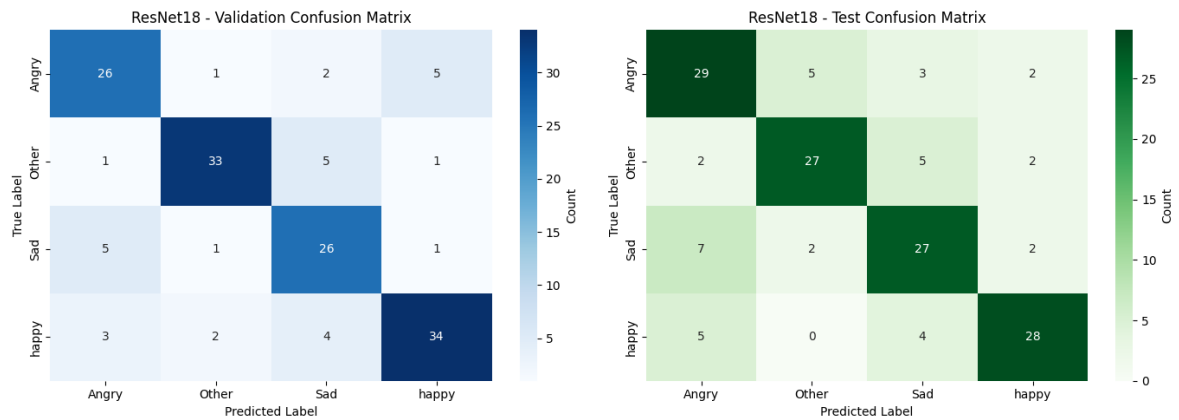
Accuracy: 0.7933
Precision: 0.7985
Recall: 0.7933
F1-Score: 0.7949
Loss: 0.9002

TEST:

Accuracy: 0.7400
Precision: 0.7445
Recall: 0.7400
F1-Score: 0.7413
Loss: 1.0402

CONFUSION MATRICES (TEXT):

CONFUSION MATRICES (HEATMAPS):



MOBILENETV2 EVALUATION RESULTS

VALIDATION:

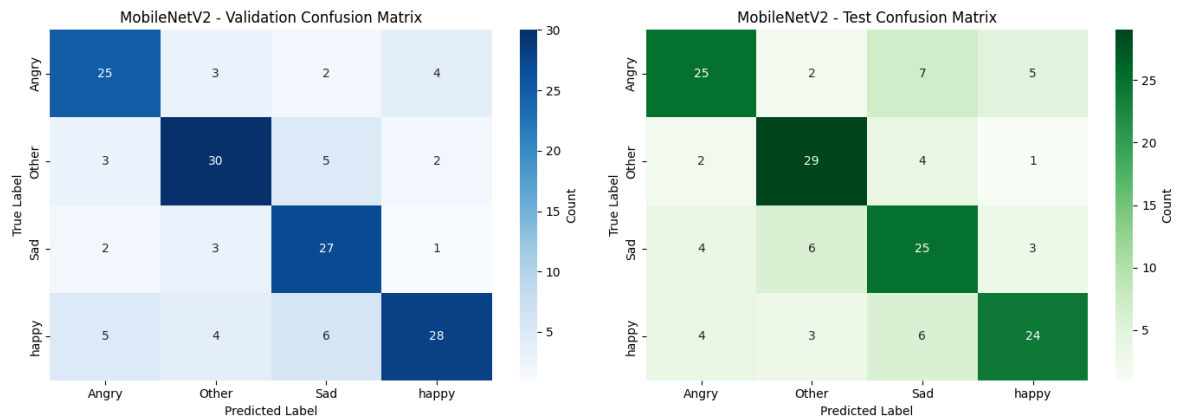
Accuracy: 0.7333
Precision: 0.7397
Recall: 0.7333
F1-Score: 0.7328
Loss: 1.0521

TEST:

Accuracy: 0.6867
Precision: 0.6899
Recall: 0.6867
F1-Score: 0.6863
Loss: 1.0913

CONFUSION MATRICES (TEXT):

CONFUSION MATRICES (HEATMAPS):



INCEPTIONV3 EVALUATION RESULTS

VALIDATION:

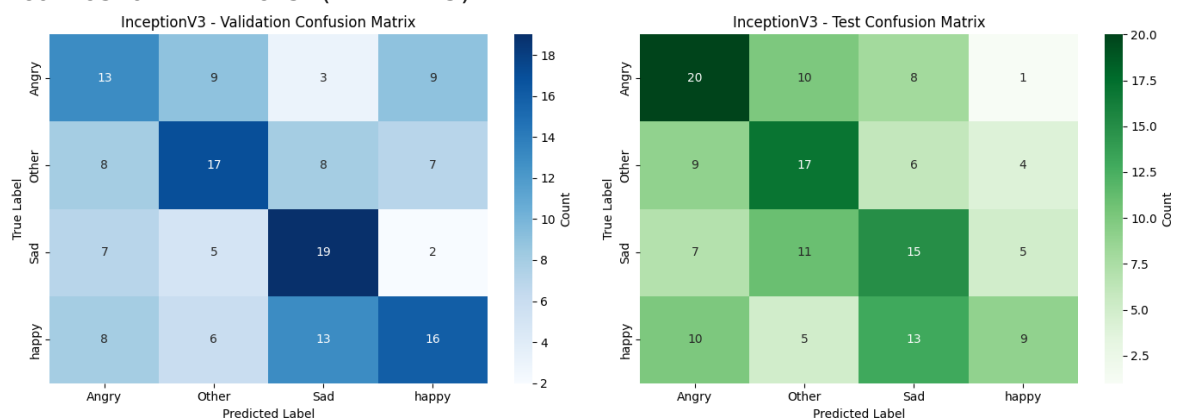
Accuracy: 0.4333
 Precision: 0.4365
 Recall: 0.4333
 F1-Score: 0.4311
 Loss: 1.3760

TEST:

Accuracy: 0.4067
 Precision: 0.4152
 Recall: 0.4067
 F1-Score: 0.3999
 Loss: 1.2979

CONFUSION MATRICES (TEXT):

CONFUSION MATRICES (HEATMAPS):



Transfer Learning

```
In [93]: # get highest accuracy among all models with its name
all_results = [
    vgg16_results,
    densenet_results,
    resnet18_results,
    mobilenetv2_results,
    inceptionv3_results
]

valid_results = [
    result for result in all_results
    if result is not None and 'test_accuracy' in result and 'model_name'
]

if not valid_results:
    raise ValueError("No valid model results found. Run the model training")

best_model_results = max(valid_results, key=lambda x: x['test_accuracy'])
best_model_name = best_model_results['model_name']

print(f"\nBest model selected: {best_model_name}")
```

```
print(f"Test Accuracy: {best_model_results['test_accuracy']:.4f}")

# load the best model for transfer learning
# each model has a different classifier layer, so we handle them separate
if best_model_name == "VGG16":
    # load pretrained VGG16 model with ImageNet weights
    transfer_model = models.vgg16(weights=models.VGG16_Weights.IMAGENET1K

    # freeze pretrained layers
    for param in transfer_model.parameters():
        param.requires_grad = False

    # replace classifier
    in_features = transfer_model.classifier[6].in_features

    transfer_model.classifier[6] = nn.Linear(in_features, num_classes)

    # train only classifier
    optimizer = torch.optim.Adam(
        transfer_model.classifier[6].parameters(),
        lr=1e-3
    )

elif best_model_name == "DenseNet121":
    transfer_model = models.densenet121(weights=models.DenseNet121_Weight

    for param in transfer_model.parameters():
        param.requires_grad = False

    in_features = transfer_model.classifier.in_features

    transfer_model.classifier = nn.Linear(in_features, num_classes)

    optimizer = torch.optim.Adam(
        transfer_model.classifier.parameters(),
        lr=1e-3
    )

elif best_model_name == "ResNet18":
    transfer_model = models.resnet18(weights=models.ResNet18_Weights.IMAG

    for param in transfer_model.parameters():
        param.requires_grad = False

    in_features = transfer_model.fc.in_features

    transfer_model.fc = nn.Sequential(
        nn.Dropout(0.3),
        nn.Linear(in_features, num_classes)
    )

    optimizer = torch.optim.Adam(
        transfer_model.fc.parameters(),
        lr=1e-3
    )

elif best_model_name == "MobileNetV2":
    transfer_model = models.mobilenet_v2(weights=models.MobileNet_V2_Weig

    for param in transfer_model.parameters():
```

```
        param.requires_grad = False

    in_features = transfer_model.classifier[1].in_features

    transfer_model.classifier[1] = nn.Linear(in_features, num_classes)

    optimizer = torch.optim.Adam(
        transfer_model.classifier[1].parameters(),
        lr=1e-3
    )

elif best_model_name == "InceptionV3":
    transfer_model = models.inception_v3(weights=models.Inception_V3_Weig

    for param in transfer_model.parameters():
        param.requires_grad = False

    in_features = transfer_model.fc.in_features

    transfer_model.fc = nn.Linear(in_features, num_classes)

    optimizer = torch.optim.Adam(
        transfer_model.fc.parameters(),
        lr=1e-3
    )

    # Override dataloaders with 299x299 for pretrained InceptionV3
    inception_train_transform = transforms.Compose([
        transforms.Resize((299, 299)),
        transforms.RandAugment(num_ops=10, magnitude=3),
        transforms.ToTensor(),
        transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.22
    ])
    inception_val_test_transform = transforms.Compose([
        transforms.Resize((299, 299)),
        transforms.ToTensor(),
        transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.22
    ])

    inception_train_dataset = CustomSubset(full_dataset, train_indices, t
    inception_val_dataset = CustomSubset(full_dataset, val_indices, t
    inception_test_dataset = CustomSubset(full_dataset, test_indices, t

    train_dataloader = DataLoader(inception_train_dataset, batch_size=BAT
    val_dataloader = DataLoader(inception_val_dataset, batch_size=BAT
    test_dataloader = DataLoader(inception_test_dataset, batch_size=BAT

    # Move model to GPU/CPU
    transfer_model = transfer_model.to(dv)

    # loss function and scheduler
    criterion = nn.CrossEntropyLoss()

    scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
        optimizer,
        mode='min',
        factor=0.5,
        patience=3
    )
```

```
# training loop for transfer learning
num_epochs = 50
best_val_accuracy = 0.0

print("\nStarting Transfer Learning Training...\n")

for epoch in range(num_epochs):

    transfer_model.train()

    train_loss = 0.0

    for images, labels in train_dataloader:

        images = images.to(dv)
        labels = labels.to(dv)

        outputs = transfer_model(images)

        # InceptionV3 returns special output object during training
        if best_model_name == "InceptionV3":
            outputs = outputs.logits

        loss = criterion(outputs, labels)

        optimizer.zero_grad()

        loss.backward()

        optimizer.step()

        train_loss += loss.item()

    avg_train_loss = train_loss / len(train_dataloader)

    # validation
    transfer_model.eval()

    val_loss = 0.0

    val_preds = []
    val_true = []

    with torch.no_grad():

        for images, labels in val_dataloader:

            images = images.to(dv)
            labels = labels.to(dv)

            outputs = transfer_model(images)

            if best_model_name == "InceptionV3":
                outputs = outputs.logits

            loss = criterion(outputs, labels)

            val_loss += loss.item()
```

```
_, predicted = torch.max(outputs, 1)

val_preds.extend(predicted.cpu().numpy())
val_true.extend(labels.cpu().numpy())

avg_val_loss = val_loss / len(val_dataloader)

val_accuracy = accuracy_score(val_true, val_preds)

# Save best model
if val_accuracy > best_val_accuracy:
    best_val_accuracy = val_accuracy
    torch.save(transfer_model.state_dict(), "best_transfer_learning_m
    print(f" → Model saved! (Val Accuracy: {val_accuracy:.4f})")

scheduler.step(avg_val_loss)

current_lr = optimizer.param_groups[0]['lr']
print(f"Epoch [{epoch+1}/{num_epochs}], Train Loss: {avg_train_loss:.
      f"Val Loss: {avg_val_loss:.4f}, Val Acc: {val_accuracy:.4f},

print("\nTransfer Learning Training Complete!")
print(f"Best Validation Accuracy: {best_val_accuracy:.4f}")
```

Best model selected: DenseNet121
Test Accuracy: 0.7667

Starting Transfer Learning Training...

→ Model saved! (Val Accuracy: 0.3933)
Epoch [1/50], Train Loss: 1.3544, Val Loss: 1.2781, Val Acc: 0.3933, LR: 0.001000
→ Model saved! (Val Accuracy: 0.4133)
Epoch [2/50], Train Loss: 1.2446, Val Loss: 1.2431, Val Acc: 0.4133, LR: 0.001000
→ Model saved! (Val Accuracy: 0.4800)
Epoch [3/50], Train Loss: 1.1693, Val Loss: 1.2213, Val Acc: 0.4800, LR: 0.001000
Epoch [4/50], Train Loss: 1.0933, Val Loss: 1.2138, Val Acc: 0.4600, LR: 0.001000
Epoch [5/50], Train Loss: 1.0752, Val Loss: 1.1687, Val Acc: 0.4733, LR: 0.001000
→ Model saved! (Val Accuracy: 0.5133)
Epoch [6/50], Train Loss: 1.0239, Val Loss: 1.1314, Val Acc: 0.5133, LR: 0.001000
Epoch [7/50], Train Loss: 0.9875, Val Loss: 1.1291, Val Acc: 0.4867, LR: 0.001000
→ Model saved! (Val Accuracy: 0.5333)
Epoch [8/50], Train Loss: 1.0042, Val Loss: 1.1304, Val Acc: 0.5333, LR: 0.001000
Epoch [9/50], Train Loss: 0.9651, Val Loss: 1.0871, Val Acc: 0.5000, LR: 0.001000
→ Model saved! (Val Accuracy: 0.5600)
Epoch [10/50], Train Loss: 0.9564, Val Loss: 1.0788, Val Acc: 0.5600, LR: 0.001000
Epoch [11/50], Train Loss: 0.9390, Val Loss: 1.0952, Val Acc: 0.5533, LR: 0.001000
Epoch [12/50], Train Loss: 0.9261, Val Loss: 1.0611, Val Acc: 0.5467, LR: 0.001000
Epoch [13/50], Train Loss: 0.9000, Val Loss: 1.0786, Val Acc: 0.4933, LR: 0.001000
Epoch [14/50], Train Loss: 0.8809, Val Loss: 1.0587, Val Acc: 0.5133, LR: 0.001000
Epoch [15/50], Train Loss: 0.8741, Val Loss: 1.0791, Val Acc: 0.4800, LR: 0.001000
Epoch [16/50], Train Loss: 0.8626, Val Loss: 1.0700, Val Acc: 0.5000, LR: 0.001000
→ Model saved! (Val Accuracy: 0.5800)
Epoch [17/50], Train Loss: 0.8431, Val Loss: 1.0142, Val Acc: 0.5800, LR: 0.001000
→ Model saved! (Val Accuracy: 0.5867)
Epoch [18/50], Train Loss: 0.8454, Val Loss: 1.0118, Val Acc: 0.5867, LR: 0.001000
Epoch [19/50], Train Loss: 0.8554, Val Loss: 1.0563, Val Acc: 0.5467, LR: 0.001000
Epoch [20/50], Train Loss: 0.8252, Val Loss: 1.0024, Val Acc: 0.5533, LR: 0.001000
→ Model saved! (Val Accuracy: 0.6067)
Epoch [21/50], Train Loss: 0.8193, Val Loss: 0.9783, Val Acc: 0.6067, LR: 0.001000
Epoch [22/50], Train Loss: 0.8180, Val Loss: 1.0195, Val Acc: 0.5800, LR: 0.001000
Epoch [23/50], Train Loss: 0.8068, Val Loss: 0.9888, Val Acc: 0.5733, LR: 0.001000

```
→ Model saved! (Val Accuracy: 0.6267)
Epoch [24/50], Train Loss: 0.8167, Val Loss: 1.0172, Val Acc: 0.6267, LR:
0.001000
Epoch [25/50], Train Loss: 0.7967, Val Loss: 1.0034, Val Acc: 0.6000, LR:
0.000500
Epoch [26/50], Train Loss: 0.8162, Val Loss: 0.9999, Val Acc: 0.5533, LR:
0.000500
Epoch [27/50], Train Loss: 0.7534, Val Loss: 0.9754, Val Acc: 0.5733, LR:
0.000500
Epoch [28/50], Train Loss: 0.7864, Val Loss: 0.9964, Val Acc: 0.5867, LR:
0.000500
Epoch [29/50], Train Loss: 0.7908, Val Loss: 0.9915, Val Acc: 0.5400, LR:
0.000500
→ Model saved! (Val Accuracy: 0.6400)
Epoch [30/50], Train Loss: 0.7587, Val Loss: 0.9572, Val Acc: 0.6400, LR:
0.000500
Epoch [31/50], Train Loss: 0.7714, Val Loss: 0.9858, Val Acc: 0.6000, LR:
0.000500
Epoch [32/50], Train Loss: 0.7539, Val Loss: 0.9615, Val Acc: 0.5733, LR:
0.000500
Epoch [33/50], Train Loss: 0.7204, Val Loss: 0.9511, Val Acc: 0.6133, LR:
0.000500
Epoch [34/50], Train Loss: 0.7826, Val Loss: 0.9606, Val Acc: 0.6000, LR:
0.000500
Epoch [35/50], Train Loss: 0.7726, Val Loss: 0.9516, Val Acc: 0.6000, LR:
0.000500
Epoch [36/50], Train Loss: 0.7123, Val Loss: 0.9717, Val Acc: 0.6000, LR:
0.000500
Epoch [37/50], Train Loss: 0.7407, Val Loss: 0.9471, Val Acc: 0.6067, LR:
0.000500
Epoch [38/50], Train Loss: 0.7547, Val Loss: 0.9380, Val Acc: 0.6133, LR:
0.000500
Epoch [39/50], Train Loss: 0.7307, Val Loss: 0.9387, Val Acc: 0.6400, LR:
0.000500
Epoch [40/50], Train Loss: 0.7419, Val Loss: 0.9411, Val Acc: 0.6067, LR:
0.000500
Epoch [41/50], Train Loss: 0.7446, Val Loss: 0.9541, Val Acc: 0.5667, LR:
0.000500
Epoch [42/50], Train Loss: 0.7139, Val Loss: 0.9289, Val Acc: 0.6267, LR:
0.000500
Epoch [43/50], Train Loss: 0.7158, Val Loss: 0.9430, Val Acc: 0.6200, LR:
0.000500
→ Model saved! (Val Accuracy: 0.6533)
Epoch [44/50], Train Loss: 0.7411, Val Loss: 0.9337, Val Acc: 0.6533, LR:
0.000500
Epoch [45/50], Train Loss: 0.7242, Val Loss: 0.9226, Val Acc: 0.6467, LR:
0.000500
Epoch [46/50], Train Loss: 0.7565, Val Loss: 0.9341, Val Acc: 0.6200, LR:
0.000500
Epoch [47/50], Train Loss: 0.7209, Val Loss: 0.9333, Val Acc: 0.6467, LR:
0.000500
Epoch [48/50], Train Loss: 0.7187, Val Loss: 0.9187, Val Acc: 0.6400, LR:
0.000500
Epoch [49/50], Train Loss: 0.7338, Val Loss: 0.9393, Val Acc: 0.5867, LR:
0.000500
Epoch [50/50], Train Loss: 0.7189, Val Loss: 0.9116, Val Acc: 0.6333, LR:
0.000500
```

Transfer Learning Training Complete!
Best Validation Accuracy: 0.6533

Evaluation (Transfer Learning)

```
In [94]: # load best transfer learning model for final evaluation
if os.path.exists("best_transfer_learning_model.pth"):
    transfer_model.load_state_dict(torch.load("best_transfer_learning_mod
    transfer_results = evaluate_model(transfer_model, f"Transfer Learning
```

TRANSFER LEARNING (DENSENET121) EVALUATION RESULTS

VALIDATION:

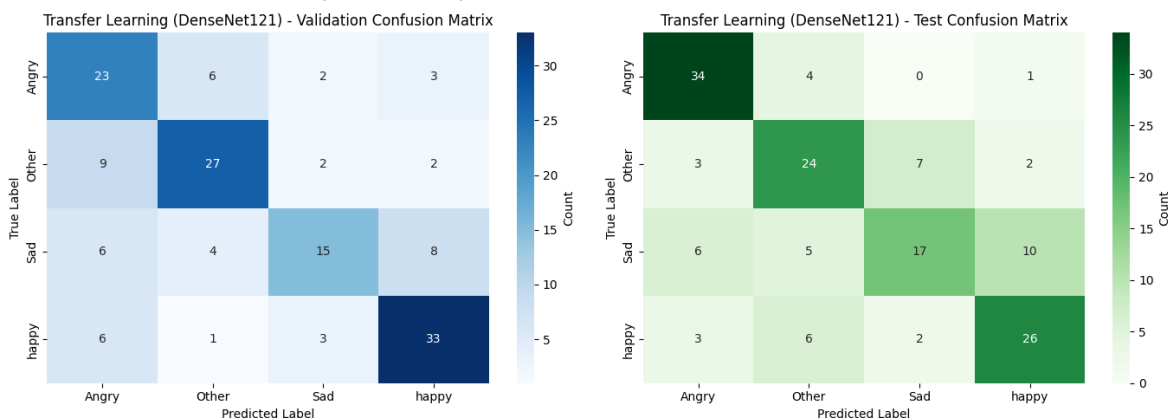
Accuracy: 0.6533
 Precision: 0.6636
 Recall: 0.6533
 F1-Score: 0.6509
 Loss: 0.9337

TEST:

Accuracy: 0.6733
 Precision: 0.6700
 Recall: 0.6733
 F1-Score: 0.6650
 Loss: 0.8762

CONFUSION MATRICES (TEXT):

CONFUSION MATRICES (HEATMAPS):



```
In [ ]: # GPU Memory Cleanup
def clear_gpu_memory():
    """
    Clear GPU memory after model evaluation.
    Useful for preventing out-of-memory errors when evaluating multiple m
    """
    if torch.cuda.is_available():
        torch.cuda.empty_cache()
        torch.cuda.reset_peak_memory_stats()
        print(f"GPU memory cleared. Current GPU memory usage: {torch.cuda
    else:
        print("CUDA not available. No GPU memory to clear.")

# Clear GPU memory after evaluation
clear_gpu_memory()
```